

Digital Signage Display Management Application

Software Design Specification: Digital Signage Display Management Application

1. High-Level Architecture & Diagrams
2. Directory & Folder Structure
3. Database Design & MongoDB/Beanie Document Models
4. API Specifications & Request/Response Models
5. Frontend Architecture & Redux State Design
6. Excel Template & Validation Rules
7. Failure Scenarios & Error Recovery Matrix
8. Performance & Security Considerations
9. Testing Strategy
10. Acceptance Criteria (Definition of Done)

Software Design Specification: Digital Signage Display Management Application

This document defines the complete functional and technical specification for the **Digital Signage Display Management Application** assignment. It serves as an implementation guide for Full-Stack Developer candidates, detailing the architecture, database schemas, API contracts, UI components, Redux state slices, validation rules, error handling strategies, and testing requirements.

1. High-Level Architecture & Diagrams

The application is structured as a decoupled SPA (Single Page Application) frontend communicating with an asynchronous REST API backend.

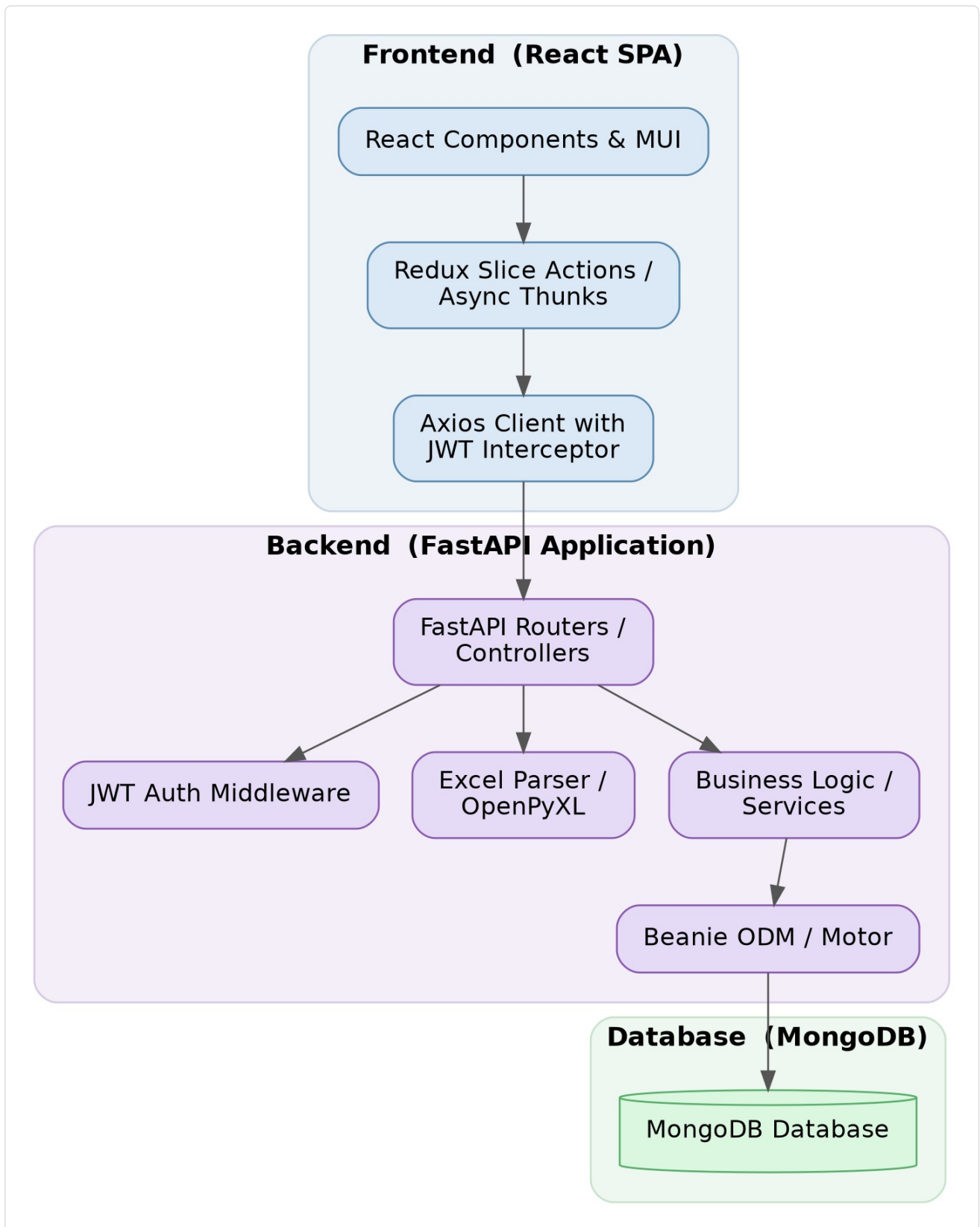


Figure 1. High-Level Architecture

1.1 Authentication & Request Flow

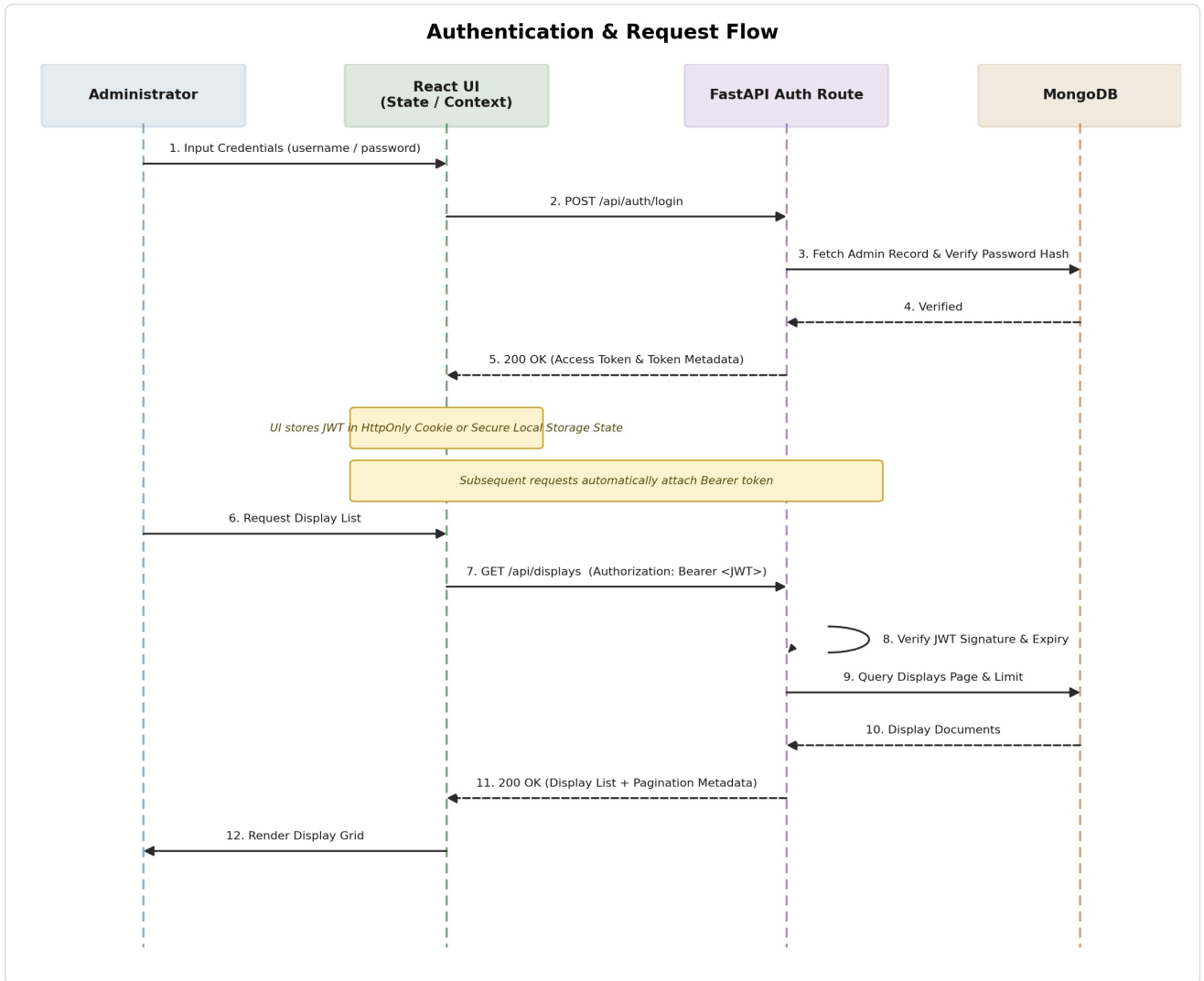
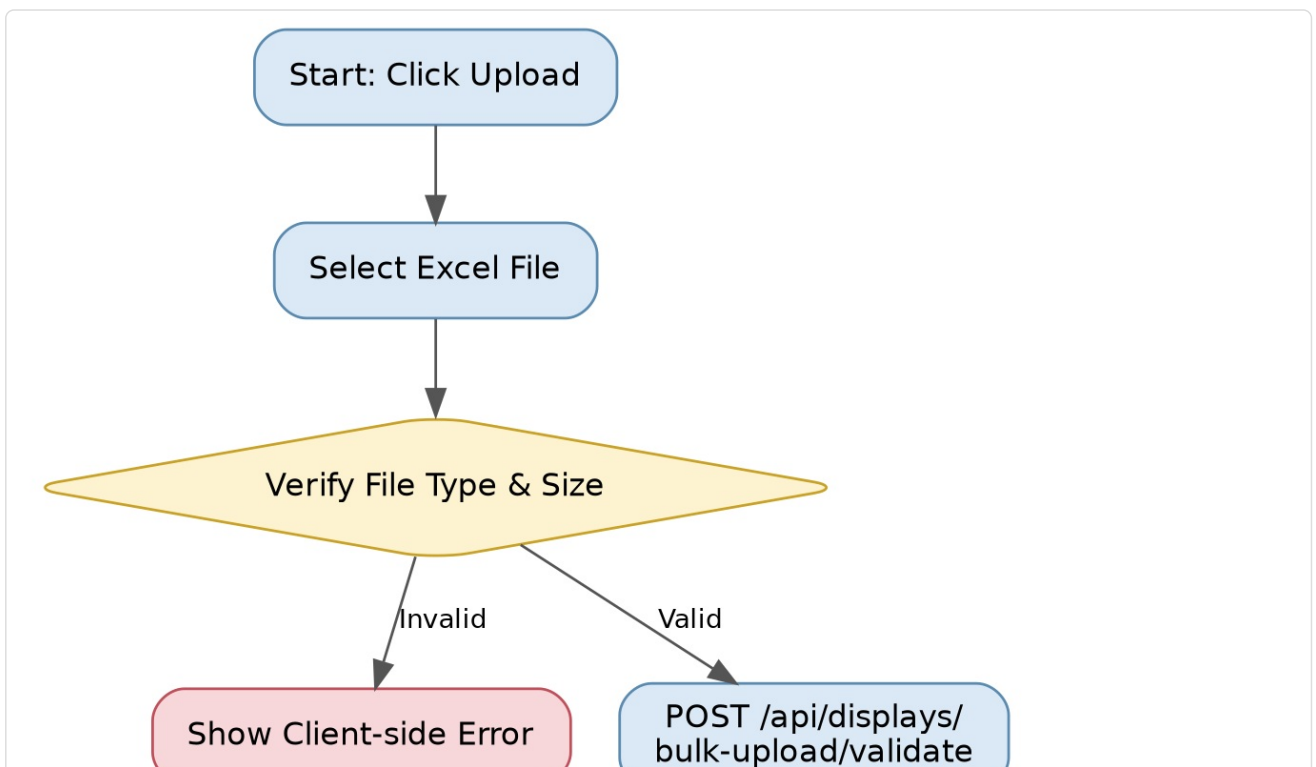


Figure 2. Authentication & Request Flow

1.2 Bulk Upload Workflow



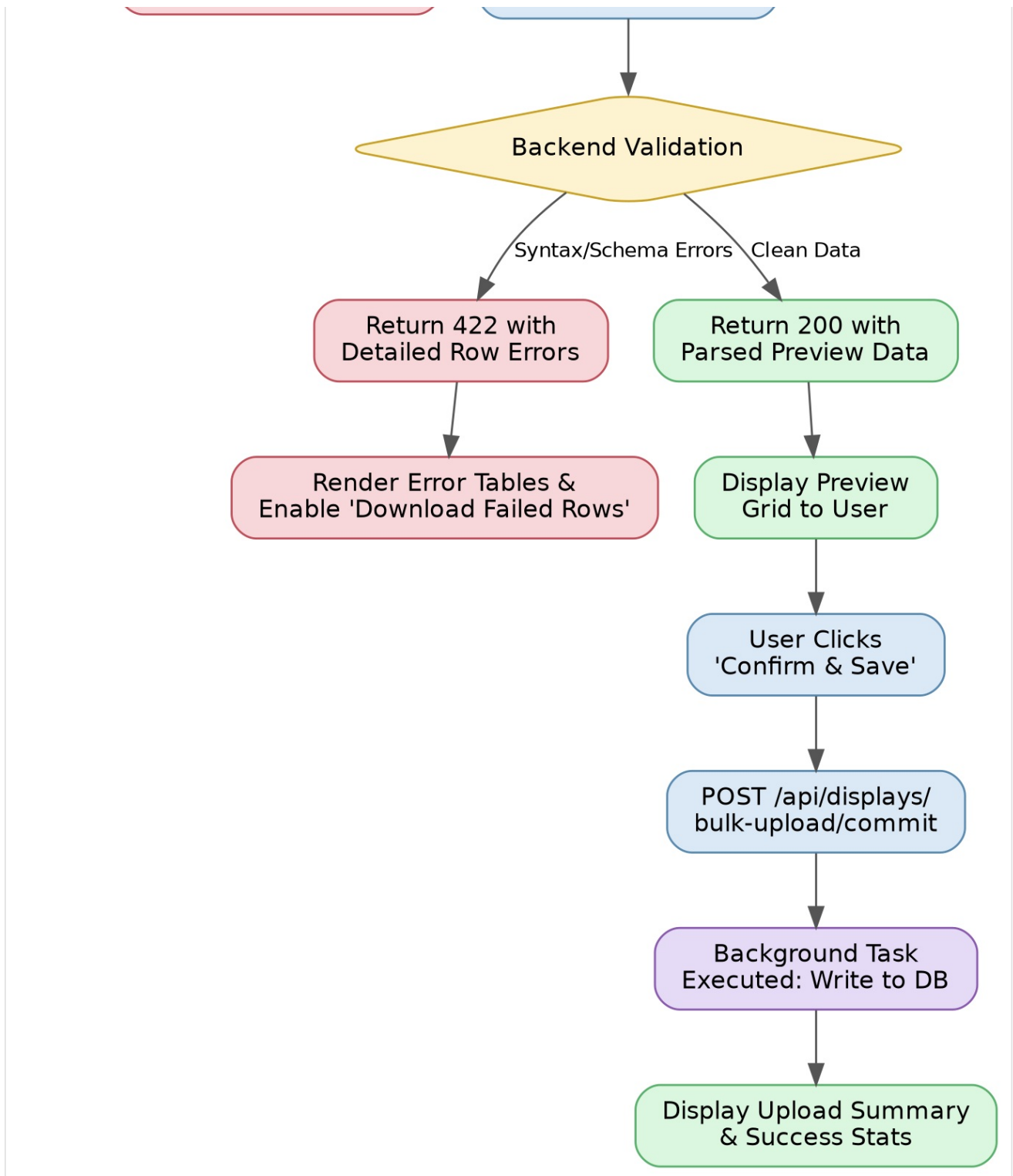


Figure 3. Bulk Upload Workflow

2. Directory & Folder Structure

A monorepo structure containing decoupled `frontend` and `backend` directories.

```

signage-management-app/
├── backend/
│   ├── app/
│   │   ├── api/
│   │   │   ├── v1/
│   │   │   │   ├── endpoints/
│   │   │   │   │   ├── auth.py
│   │   │   │   │   ├── displays.py
│   │   │   │   │   └── content_profiles.py
│   │   │   │   └── api.py
│   │   │   └── deps.py
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── security.py
│   │   │   └── database.py
│   │   ├── models/
│   │   │   ├── display.py
│   │   │   ├── content_profile.py
│   │   │   └── admin.py
│   │   ├── schemas/
│   │   │   ├── auth.py
│   │   │   ├── display.py
│   │   │   ├── content_profile.py
│   │   │   └── excel.py
│   │   ├── services/
│   │   │   ├── excel_parser.py
│   │   │   └── display_service.py
│   │   └── main.py
│   ├── tests/
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/
│   ├── public/
│   │   └── assets/
│   ├── src/
│   │   ├── assets/
│   │   ├── components/
│   │   │   ├── common/
│   │   │   │   ├── ProtectedRoute.jsx
│   │   │   │   └── ConfirmDialog.jsx
│   │   │   └── displays/
│   │   │       ├── DisplayCard.jsx
│   │   │       ├── DisplayForm.jsx
│   │   │       ├── DisplayTable.jsx
│   │   │       └── BulkUploadPreview.jsx
│   │   ├── pages/
│   │   │   ├── LoginPage.jsx
│   │   │   ├── DisplaysPage.jsx
│   │   │   └── DisplayDetailPage.jsx
│   │   ├── redux/
│   │   │   ├── slices/
│   │   │   │   ├── authSlice.js
│   │   │   │   ├── displaySlice.js
│   │   │   │   └── uploadSlice.js
│   │   │   └── store.js
│   │   ├── services/
│   │   │   ├── api.js
│   │   │   └── authService.js
│   │   ├── utils/
│   │   │   └── validators.js
│   │   ├── App.jsx
│   │   ├── index.css
│   │   └── main.jsx
│   ├── package.json
│   └── vite.config.js

```

3. Database Design & MongoDB/Beanie Document Models

We utilize **MongoDB** as our document store, interfaced using **Beanie ODM** (which relies on Pydantic and Motor).

3.1 Schema Decisions & Relationships

- **Display Document (`displays`)**: Stored as a main document.
- **Content Profile Document (`content_profiles`)**: Stored as a referenced model. A display can have multiple

content profiles. We store referenced IDs (`Link[ContentProfile]`) in the Display document to enforce normalization, as Content Profiles can be shared across multiple displays or managed independently.

- **Indexes:**
 - Unique index on `display_id` (an alphanumeric human-readable identifier).
 - Text index on `name`, `ip_address`, and `location` for fast multi-field search queries.

3.2 Database Relationship Diagram

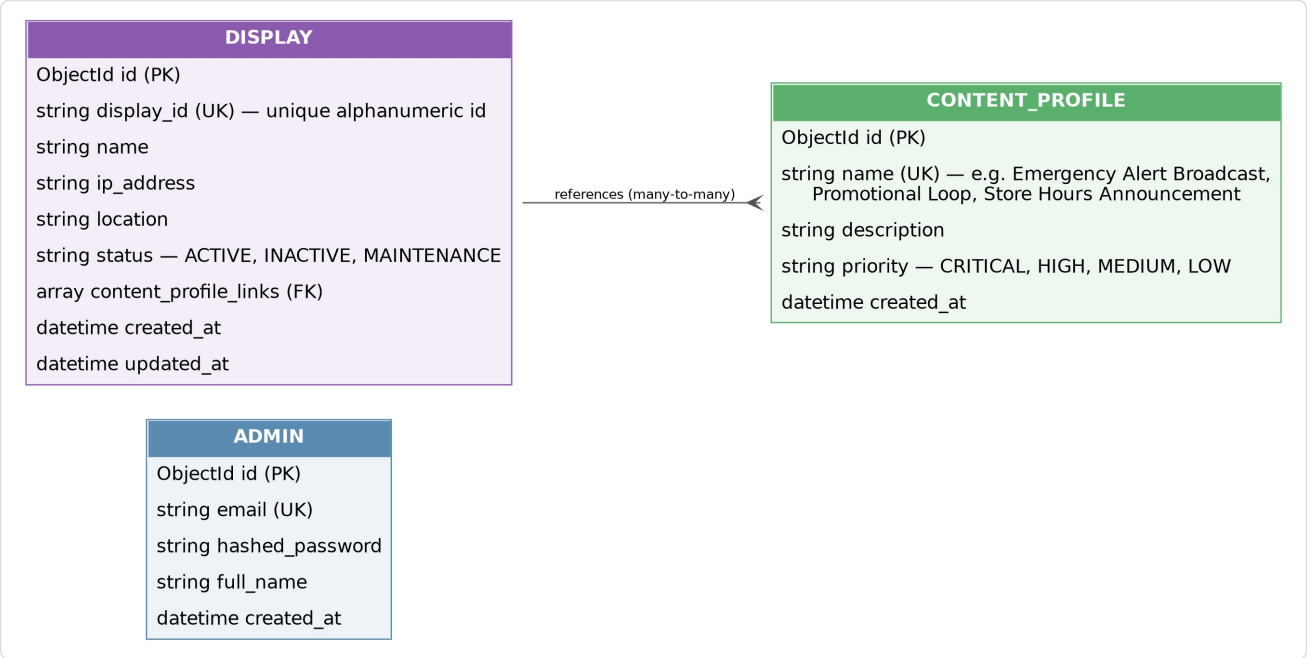


Figure 4. Database Relationship Diagram

3.3 Beanie Document Models

```
# backend/app/models/content_profile.py
from datetime import datetime
from typing import Optional
from beanie import Document
from pydantic import Field

class ContentProfile(Document):
    name: str = Field(..., unique=True, min_length=2, max_length=50)
    description: Optional[str] = Field(None, max_length=250)
    priority: str = Field("MEDIUM", regex="^(CRITICAL|HIGH|MEDIUM|LOW)$")
    created_at: datetime = Field(default_factory=datetime.utcnow)

    class Settings:
        name = "content_profiles"
        indexes = ["name"]

# backend/app/models/display.py
from datetime import datetime
from typing import List, Optional
from beanie import Document, Link
from pydantic import Field, IPvAnyAddress
from app.models.content_profile import ContentProfile

class Display(Document):
    display_id: str = Field(..., unique=True, min_length=3, max_length=30)
    name: str = Field(..., min_length=2, max_length=100)
    ip_address: str = Field(..., description="IPv4 or IPv6 address of the display's media player")
    location: str = Field(..., min_length=2, max_length=150)
    status: str = Field("ACTIVE", regex="^(ACTIVE|INACTIVE|MAINTENANCE)$")
    content_profiles: List[Link[ContentProfile]] = Field(default=[])
    created_at: datetime = Field(default_factory=datetime.utcnow)
    updated_at: datetime = Field(default_factory=datetime.utcnow)

    class Settings:
        name = "displays"
        indexes = [
            "display_id",
            [
                ("name", "text"),
                ("location", "text"),
                ("ip_address", "text")
            ]
        ]
```

4. API Specifications & Request/Response Models

All endpoints require JWT authorization, unless specified otherwise. JWT must be attached as a Bearer token:

Authorization: Bearer <token>.

4.1 Authentication Endpoints

POST /api/auth/login

- **Purpose:** Authenticate administrator and issue JWT.
- **Authentication:** None.
- **Request Body:**

```
{
  "username": "admin@example.com",
  "password": "SecurePassword123"
}
```

- **Success Response (200 OK):**


```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ0eSIsImVpdCI6IjIwMjYtMDctMjJlMTQ6MjM6MTQZ",
  "token_type": "bearer",
  "expires_at": "2026-07-22T14:23:14Z"
}
```

- **Failure Responses:**
 - **401 Unauthorized:** Invalid credentials.
-

4.2 Display Management Endpoints

GET `/api/displays`

- **Purpose:** Retrieve a paginated list of displays with optional search/filter.
- **Query Parameters:**
 - `page` (int, default: 1): Page number.
 - `limit` (int, default: 10): Items per page.
 - `search` (str, optional): Search text matching display name, IP, or location.
 - `status` (str, optional): Filter by `ACTIVE`, `INACTIVE`, or `MAINTENANCE`.
- **Success Response (200 OK):**

```
{
  "data": [
    {
      "id": "60d5ec49f1b29a2e6840723a",
      "display_id": "DSP-001",
      "name": "Main Entrance Lobby Screen",
      "ip_address": "192.168.1.50",
      "location": "Mall Concourse, Entrance 1",
      "status": "ACTIVE",
      "content_profiles": [
        {
          "id": "60d5ec49f1b29a2e68407211",
          "name": "Emergency Alert Broadcast",
          "priority": "HIGH"
        }
      ],
      "created_at": "2026-07-22T10:00:00Z"
    }
  ],
  "pagination": {
    "total": 124,
    "page": 1,
    "limit": 10,
    "pages": 13
  }
}
```

POST `/api/displays`

- **Purpose:** Create a single display.
- **Request Body:**

```
{
  "display_id": "DSP-002",
  "name": "Food Court Menu Board North",
  "ip_address": "192.168.10.12",
  "location": "HQ Block, Floor 2",
  "status": "ACTIVE",
  "content_profiles": ["Emergency Alert Broadcast", "Promotional Content Loop"]
}
```

- **Success Response (201 Created):**
 - Returns created Display document with populated content profile links.

- **Failure Responses:**

- **400 Bad Request:** Duplicate `display_id` or invalid validation schema.

PUT `/api/displays/{id}`

- **Purpose:** Edit an existing display's details.
- **Request Body:** (Same fields as POST, all optional).
- **Success Response (200 OK):** Updated display object.

DELETE `/api/displays/{id}`

- **Purpose:** Delete a display.
 - **Success Response (204 No Content).**
-

4.3 Bulk Upload Endpoints

GET `/api/displays/bulk-upload/template`

- **Purpose:** Download empty Excel template (`.xlsx`) with predefined columns and validation rules.
- **Success Response:** File attachment stream (`application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`).

POST `/api/displays/bulk-upload/validate`

- **Purpose:** Parse Excel, check syntax, structure, data-types, reference validity, and return a preview with highlightable errors.
- **Request:** Multipart Form Data with field `file` (Excel file).
- **Success Response (200 OK):**

```

{
  "is_valid": false,
  "summary": {
    "total_rows": 5,
    "valid_rows": 3,
    "failed_rows": 2
  },
  "preview": [
    {
      "row_number": 2,
      "is_valid": true,
      "data": {
        "display_id": "DSP-101",
        "name": "Loading Dock Screen 1",
        "ip_address": "192.168.1.80",
        "location": "Dock Zone B",
        "status": "ACTIVE",
        "content_profiles": ["Promotional Content Loop"]
      },
      "errors": []
    },
    {
      "row_number": 3,
      "is_valid": false,
      "data": {
        "display_id": "DSP-101",
        "name": "Loading Dock Screen 2",
        "ip_address": "999.999.9.9",
        "location": "Dock Zone C",
        "status": "INVALID_STATUS",
        "content_profiles": ["Unknown Profile"]
      },
      "errors": [
        {"field": "display_id", "message": "Duplicate display_id encountered in same sheet"},
        {"field": "ip_address", "message": "Invalid IP address format"},
        {"field": "status", "message": "Value must be ACTIVE, INACTIVE, or MAINTENANCE"},
        {"field": "content_profiles", "message": "Content profile 'Unknown Profile' will be auto-created but warning flag"}
      ]
    }
  ]
}

```

POST /api/displays/bulk-upload/commit

- **Purpose:** Save the validated preview data to the database (only creates valid rows or runs an all-or-nothing transaction based on implementation choice).
- **Request Body:** List of validated objects to commit.
- **Success Response (200 OK):** Upload statistics report.

5. Frontend Architecture & Redux State Design

The client relies on **Vite**, **React**, **React Router**, **MUI v5**, **Redux Toolkit**, and **React Hook Form**.

Component Hierarchy:

```

App
├── ProtectedRoute
│   └── Switch
│       ├── Route: /login — LoginPage
│       ├── Route: /displays — DisplaysPage (Grid, Search, Actions)
│       │   ├── Component: DisplayTable
│       │   ├── Component: DisplayFormDialog (Add/Edit Modal)
│       │   └── Component: ConfirmDeleteDialog
│       ├── Route: /displays/:id — DisplayDetailPage
│       └── Route: /displays/upload — BulkUploadPage
│           ├── Component: FileDropzone
│           ├── Component: UploadPreviewTable (Highlights Errors in Red)
│           └── Component: UploadSummaryPanel

```

5.1 Redux Slices

1. authSlice

• **State:**

```
{
  isAuthenticated: false,
  token: null,
  user: null,
  loading: false,
  error: null
}
```

• **Thunks:** loginAdmin , logoutAdmin .

2. displaySlice

• **State:**

```
{
  items: [],
  total: 0,
  currentPage: 1,
  limit: 10,
  searchQuery: "",
  selectedDisplay: null,
  loading: false,
  error: null
}
```

• **Thunks:** fetchDisplays , fetchDisplayById , createDisplay , updateDisplay , deleteDisplay .

3. uploadSlice

• **State:**

```
{
  previewData: [],
  validationSummary: null,
  isValidating: false,
  isCommitting: false,
  uploadError: null,
  failedRowsFile: null // Blobs/URLs representing downloadable failed rows Excel
}
```

• **Thunks:** validateExcelFile , commitBulkUpload .

6. Excel Template & Validation Rules

6.1 Column Specifications

Column Name	Required	Data Type	Validation Rules / Allowed Values	Default Value	Description
display_id	Yes	String	Min 3 chars, alphanumeric. Unique in DB and Excel.	None	Unique Identifier (e.g. DSP-001)
name	Yes	String	Min 2, Max 100 characters.	None	Descriptive name
ip_address	Yes	String	Must match standard IPv4 or IPv6 pattern.	None	Network address for the display's media player
location	Yes	String	Min 2, Max 150 characters.	None	Location designation
status	No	String	Choice of: ACTIVE , INACTIVE , MAINTENANCE	ACTIVE	Display status

content_profiles	No	String	Comma-separated list of content profiles.	None	Content profiles to associate (e.g. "Emergency Alert, Promotional Loop")
------------------	----	--------	---	------	--

6.2 Sheet Parsing & Backend Validation Process

- File Constraint Check:** Must be a `.xlsx` file, max size 10MB.
- Structural Validation:** Verify required sheets and columns exist. If columns are missing, fail parsing immediately.
- Row-by-Row Evaluation:**
 - Skip completely empty rows.
 - Validate fields against Pydantic validators.
 - Cross-reference: Track `display_id`s in a set during loop iteration to catch intra-sheet duplicates.
 - Check MongoDB database for existing `display_id` values. If present, flag as "Duplicate display_id in database".
 - For the `content_profiles` column, split by comma, trim whitespace. If the content profile name does not exist in the database, flag it for auto-creation or validation warning depending on configuration.

7. Failure Scenarios & Error Recovery Matrix

Scenario / Error	HTTP Status	Response Payload	User-Facing Action / UI State	Recovery Strategy
Invalid Credentials	401	<code>{"detail": "Incorrect username or password"}</code>	Highlight login form inputs in red with error helper text.	Prompt user to check credentials and try again.
Token Expired	401	<code>{"detail": "Token expired"}</code>	Redirect user to Login page with info toast.	Clear store auth state and prompt login.
Duplicate Display ID	400	<code>{"detail": "Display ID already exists"}</code>	Display error banner next to "Display ID" field in Edit/Add dialog.	Prompt user to choose a different ID.
Excel Column Missing	422	<code>{"detail": "Missing column: display_id"}</code>	Display explicit error card above dropzone specifying the missing columns.	User must fix template structure and re-upload.
Cell Format/Type Mismatch	422	Detailed row validation list.	Table row showing red highlight and hover tooltips showing error.	User edits file or uses the 'Download Failed Rows' button.
DB Outage	503	<code>{"detail": "Database connection timeout"}</code>	Global Error boundary page or Snackbar banner message.	Implement 3-retry attempt in Axios interceptor, then display "Server is temporarily busy. Retry later."

8. Performance & Security Considerations

8.1 Performance Optimization

- Bulk Database Inserts:** When committing the Excel data, avoid writing to the database cell-by-cell. Parse all items, validate them in memory, and commit using Beanie/Motor `insert_many()` batch operations.
- Pagination:** Always paginate the database query on `GET /api/displays` using `skip` and `limit`.
- Async Parsing:** Use Python's `asyncio` loop execution for parsing heavier xlsx files to ensure the single-threaded event loop of FastAPI is not blocked.

8.2 Security & Hardening

- **Password Hashing:** Store passwords using `bcrypt` (via `passlib`).
 - **JWT Lifecycle:** Sign JWT with `HS256`, use a secure environment-stored secret key, and set expiration strictly to 8 hours (`480 minutes`).
 - **Sanitize Inputs:** Escape and validate string parameters on the backend (using Pydantic features) to avoid database and HTML injection vectors.
 - **Excel Safeguards:** Limit uploaded file sizes to 10MB to prevent Denial of Service (DoS) attacks via decompression bomb files.
-

9. Testing Strategy

Candidates should implement and verify their solution using the following test matrix:

1. **Unit Tests (Backend):**
 - Excel parsing logic with mock valid and invalid `xlsx` payloads.
 - Password hashing utility validation.
 2. **API Integration Tests (FastAPI + Motor):**
 - Auth flow (`/login` with correct and incorrect credentials).
 - Protected access endpoints with and without Bearer headers.
 - CRUD endpoints with constraints (e.g. creating a display with duplicate ID returns 400).
 3. **State Management Tests (Redux):**
 - Verify store actions dispatch correctly during pending, fulfilled, and rejected statuses.
 4. **UI Tests:**
 - Check validation error highlights on Excel Upload preview table.
 - Verify responsiveness of grid cards/tables on mobile and tablet viewport widths.
-

10. Acceptance Criteria (Definition of Done)

- **Auth:** Log in successfully, retrieve JWT, attach JWT to all subsequent requests, log out clears credentials.
 - **CRUD:** Administrator can create, read, update, and delete displays. Search updates results list in real-time.
 - **Bulk Upload:**
 - Clean template downloads successfully.
 - Uploading a valid Excel file showcases preview and allows immediate commit.
 - Uploading an invalid Excel file flags exact lines, showing inline errors.
 - Allows downloading a generated spreadsheet showing only failed records with error reasons appended.
 - **Quality:** The app is completely responsive, contains empty states, loading indicators, and handles server failures gracefully. Code is clean, modular, and does not contain hardcoded credentials.
-

End of Design Specification.